

Transformers

Lesson 4: Layer Tricks, Speedups, and Interpretability

Agenda

- **Part I — Layer tricks** — making depth trainable.
- **Part II — Speedups** — making attention fast.
- **Part III — Interpretability** — what is the model actually telling you.

Learning Objectives

By the end of this lesson, you'll be able to:

- Explain the layer tricks that make deep Transformers trainable.
- Describe the main strategies for making attention faster and more efficient at inference.
- Discuss what interpretability methods can — and cannot — tell us about model behavior.

Part I — The tricks that make depth trainable

From architecture to trainability

Lesson 2 showed us **what a Transformer block contains**. This section asks a different question: **what makes it possible to stack many blocks and train them reliably?**

Residual connections · LayerNorm · FFN

We will also add one training-side companion: **label smoothing**.

What we will open

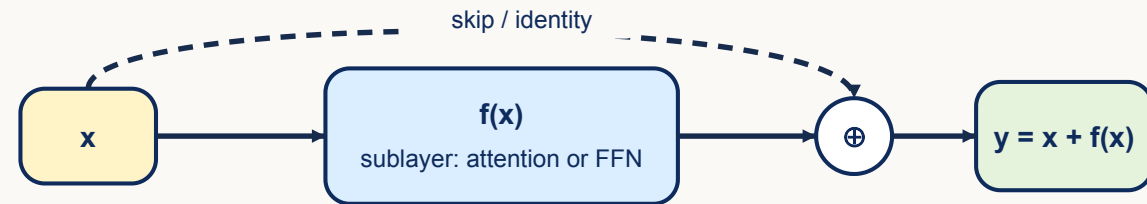
- **Residual connections** — preserve a direct path through depth.
- **LayerNorm** — keep token representations on a stable scale.
- **Feed-forward networks** — add per-token nonlinear capacity.
- **Label smoothing** — soften the training target.

(a) Residual connections — add, don't replace

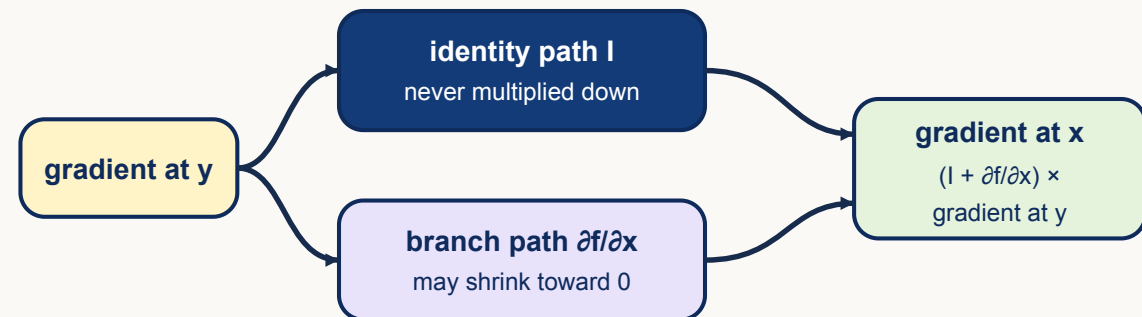
Each sublayer computes $x \mapsto x + \text{Sublayer}(x)$: it **adds its output back to its input** instead of overwriting it. The idea is imported wholesale from **ResNet** (He et al., 2016).

The book's phrasing: it "*adds more paths where the gradient can flow.*" Each sublayer now only has to learn a small **refinement** of the residual stream, not a full re-encoding. It adds no new parameters — just an elementwise sum.

forward — the sublayer refines, it does not replace



backward — the gradient arrives by TWO paths



The gradient highway, derived

With a residual sublayer $y = x + f(x)$, differentiate the output with respect to the input:

$$\frac{\partial y}{\partial x} = I + \frac{\partial f}{\partial x} \quad (1)$$

1. Backprop through a deep stack is a **product of Jacobians**. Without the skip that product is $\prod_l \partial f_l / \partial x$ — small factors collapse it toward 0: the **vanishing gradient** of deep RNNs (Module 1).
2. With the skip, every factor is $(I + \partial f_l / \partial x)$. Even if $\partial f_l / \partial x \approx 0$, expanding the product still leaves a pure $I \cdot I \cdots I$ term — **an undiluted path back to layer 1**.
3. The identity term cannot be multiplied down, so **gradients reach the earliest layers intact**.

(b) Layer normalization — a stable scale per token

Normalize **each token's activation vector across its own d_{model} features**, then rescale and shift with learned γ, β :

$$\begin{aligned}\mu &= \frac{1}{d} \sum_{i=1}^d x_i, & \sigma^2 &= \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \\ \hat{x} &= \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, & \text{LN}(x) &= \gamma \odot \hat{x} + \beta\end{aligned}\tag{2}$$

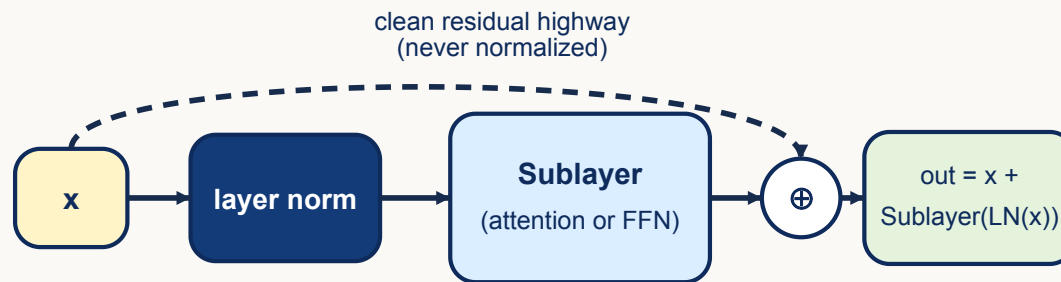
1. μ, σ^2 use **the features of one token**; no other token or example is involved.
2. $\hat{x}$ has mean 0 and variance 1; ϵ only prevents division by zero.
3. Learned $\gamma, \beta \in \mathbb{R}^{d_{\text{model}}}$ can rescale and re-centre the result.

Why it helps: every sublayer receives a predictable scale, even with variable-length sequences or one-token-at-a-time decoding.

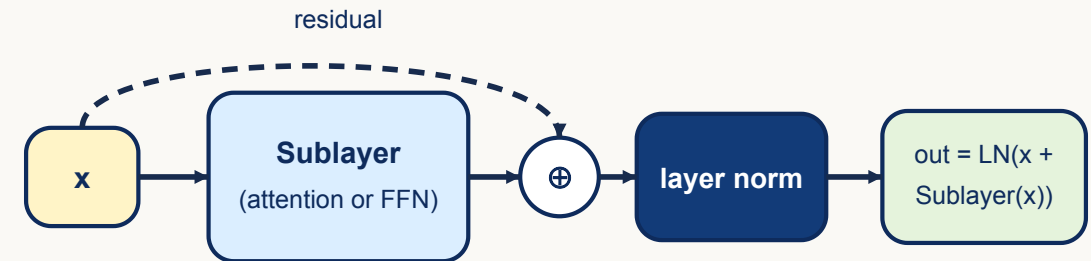
(c) Pre-LN vs post-LN — a design question, not a settled fact

Same LN, same residual. The only question is **where the LN box sits** — and it shapes how comfortably a deep stack trains.

PRE-LN — GPT-2 onward (modern default)



POST-LN — Vaswani 2017 (needs LR warmup)



Why the placement matters

Placement	Update	Behavior
Post-LN (Vaswani 2017)	$x \mapsto \text{LN} (x + \text{Sublayer}(x))$	LN <i>on</i> the residual path; the stream is renormalized every block. Needs learning-rate warmup ; gradients can be large early in training.
Pre-LN (GPT-2 onward)	$x \mapsto x + \text{Sublayer} (\text{LN}(x))$	LN <i>inside</i> the branch; the residual highway stays clean . Gradients are better-scaled at initialization → more stable, more warmup-tolerant.

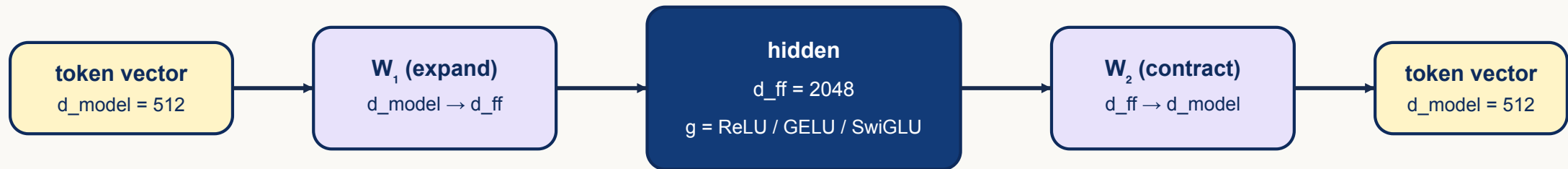
Xiong et al. (2020): pre-LN starts with **better-scaled gradients**; post-LN often needs warmup.

(d) The position-wise feed-forward network

A 2-layer MLP applied **identically and independently to every position**:

$$\text{FFN}(x) = W_2 g(W_1 x + b_1) + b_2 \quad (3)$$

applied per token — the same W_1, W_2 at every position

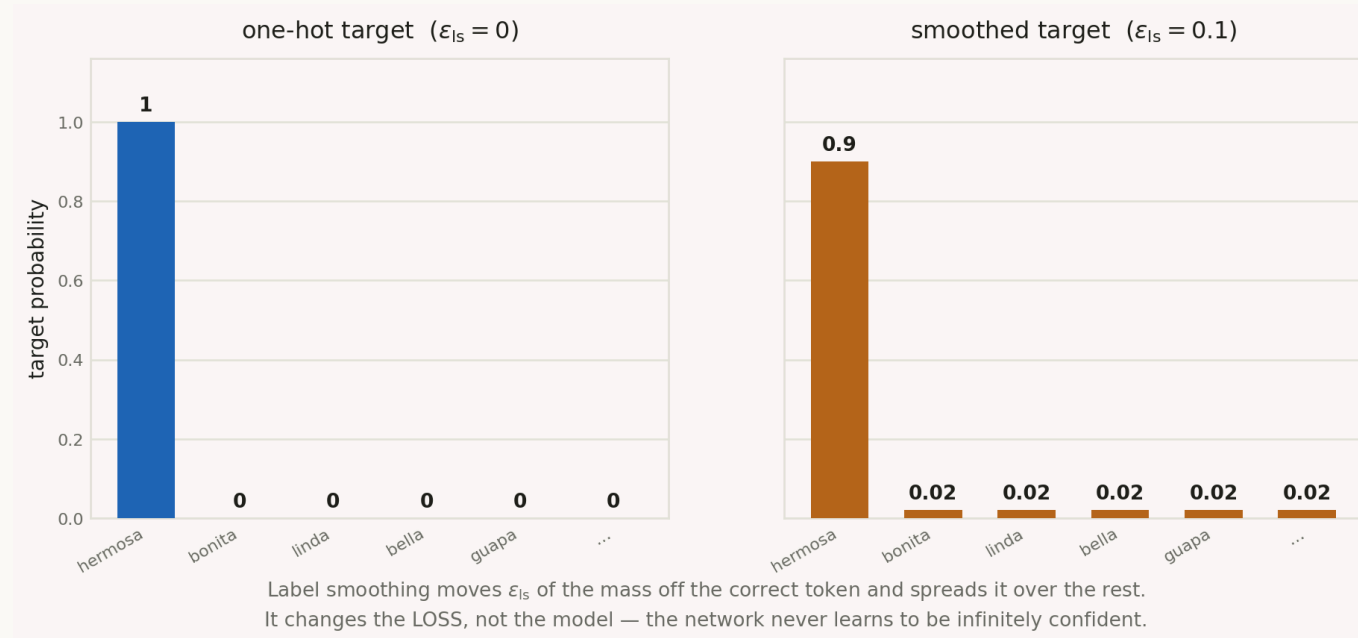


Expand → activate → contract, typically $d_{\text{ff}} = 4 d_{\text{model}}$ — no token index appears anywhere in (3): the same W_1, W_2, b_1, b_2 hit every position separately, like a 1×1 convolution. This gives each token nonlinear capacity without mixing tokens or changing sequence length.

(e) Label smoothing — soften the target, not the model

The output head trains with cross-entropy, and the target is normally **one-hot**. Label smoothing moves a small mass ϵ_{ls} off the correct class:

$$y_{\text{correct}} = 1 - \epsilon_{ls}, \quad y_{\text{other}} = \frac{\epsilon_{ls}}{|V| - 1} \quad (4)$$



Label smoothing is a *loss-side* trick

It changes the **training target distribution** — and **nothing in the forward pass moves**. The architecture, the parameter shapes and the inference-time computation are all untouched; only the cross-entropy loss sees a softened target instead of a one-hot.

Effect: the model is deliberately made *less certain*.

! Misconception: *"label smoothing modifies the model or its outputs."* It modifies only what the loss compares against **during training**. Ship the model and there is nothing different to ship.

Part II — Making attention fast, and cheap at inference

The $O(n^2)$ wall

Self-attention forms the **full** $n \times n$ score matrix before the softmax, $A = \text{softmax}(QK^\top / \sqrt{d_k})$ — every token scored against every token (Lesson 1). So **double the context, quadruple the scores**.

- **Sparse attention** — change **which** scores you compute.
- **Low-rank attention** — change the **rank** of the attention computation.
- **FlashAttention** — change **how** you compute the exact same attention.

⚠ Misconception: *"Longer context is just a bigger number."* It is a **quadratic** cost in both compute and memory — the entire reason this part exists.

Made concrete: the wall in numbers

How many attention scores does **one head, in one layer** compute?

Context n	n^2 scores	vs the row above
512	262,144 $\approx$ 0.26M	—
4,096	16,777,216 $\approx$ 16.8M	8× longer $\rightarrow$ 64× more
32,768	1,073,741,824 $\approx$ 1.07B	8× longer $\rightarrow$ 64× more

And a model is not one head: BERT-Base runs $h = 12$ heads in each of $N = 12$ layers — **144** score matrices per forward pass. At $n = 4,096$ that is $144 \times 16.8\text{M} \approx$ **2.4 billion** scores, for a single sequence.

(a) Sparse attention — attend to a structured subset

Longformer: a **local sliding window** (each token attends to its w near neighbours) plus a few **global** tokens (e.g. [CLS]) that attend to everything. Cost: $O(n \cdot w)$ — linear in n .

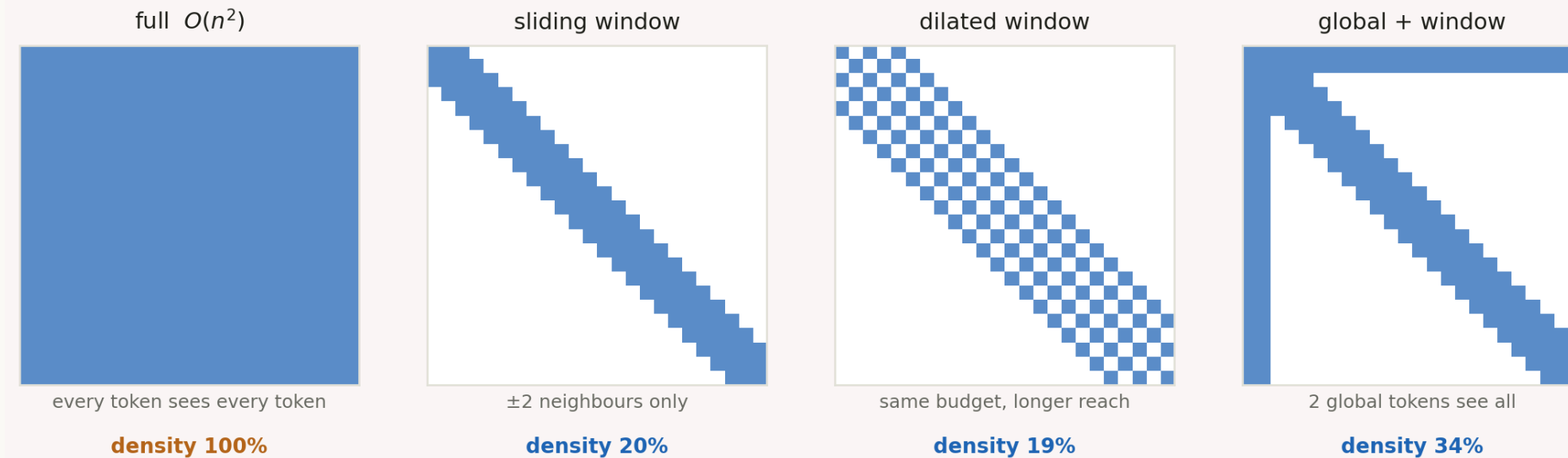
Reformer: *discover* the pattern instead of fixing it — bucket similar queries/keys by **LSH** (Module 1's locality-sensitive hashing) and attend only within a bucket. Cost: $O(n \log n)$.

Why use it: sparse patterns reduce compute and memory on long sequences without forming the full $n \times n$ matrix.

! Misconception: "Sparse attention always loses accuracy." For many tasks, locality plus a few global tokens is enough — a task-dependent trade-off, not a strict loss.

Sparse attention — patterns at a glance

Sparse attention: keep the pattern, drop the cells — rows = queries, cols = keys



Blue cells are attention scores we keep; **white cells** are never computed. A local or dilated window cuts most scores, while global tokens preserve a few long-range connections.

(b) Low-rank attention — avoid the full $n \times n$ table

In full attention, $Q, K, V \in \mathbb{R}^{n \times d}$ and

$$QK^{\top} : n \times d \longrightarrow n \times n$$

The goal is to keep every query, but avoid comparing it with a full table of all keys.

- **Linformer**: compresses the keys and values along the sequence dimension:
 $K, V : n \times d \rightarrow k \times d$ with $k \ll n$. The score table becomes $n \times k$; Q still has one row per token.
- **Performer**: maps Q and K to a smaller feature space and reorders the products, so it never creates the $n \times n$ table:

$$\text{Attention}(Q, K, V) \approx \phi(Q)(\phi(K)^{\top}V).$$

For $n = 1000$ and $k = 100$, Linformer replaces roughly **1,000,000** score pairs with **100,000**. Both methods are **approximations** that can approach linear cost when the smaller

(c) FlashAttention — the bottleneck is memory traffic

On a GPU the scarce resource is not arithmetic, it is **moving data**. The $n \times n$ scores must travel between two very different memories:

HBM

high-bandwidth memory — big and
slow: ~40 GB at ~1 TB/s

SRAM

on-chip memory — tiny and fast:
~20 MB at ~20 TB/s

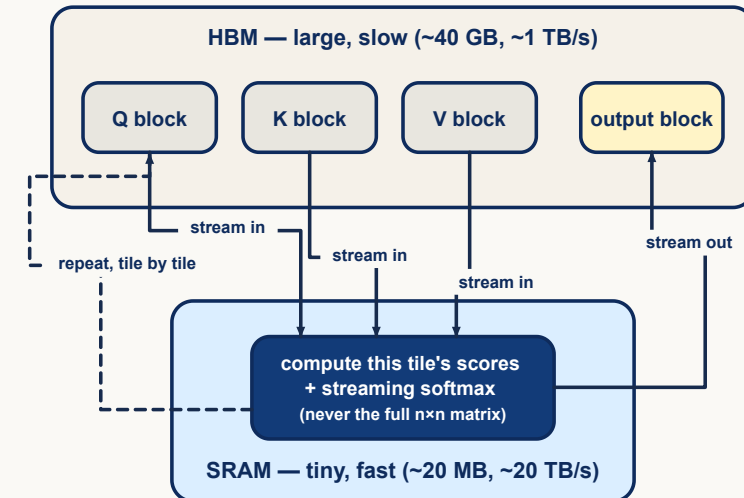
SRAM is about **20× faster** but roughly **2,000× smaller**. A classic implementation writes the whole score matrix to HBM and reads it back — so the kernel spends its life waiting on memory, not computing.

FlashAttention — tiling and recomputation

Tiling — split Q , K , V into blocks small enough for SRAM and compute attention block by block, combining pieces with an **online (streaming) softmax**. The huge $n \times n$ matrix is **never** written to HBM.

Recomputation — don't store the scores for the backward pass; **recompute** them on the fly in SRAM. More FLOPs, far less memory traffic — net, much faster.

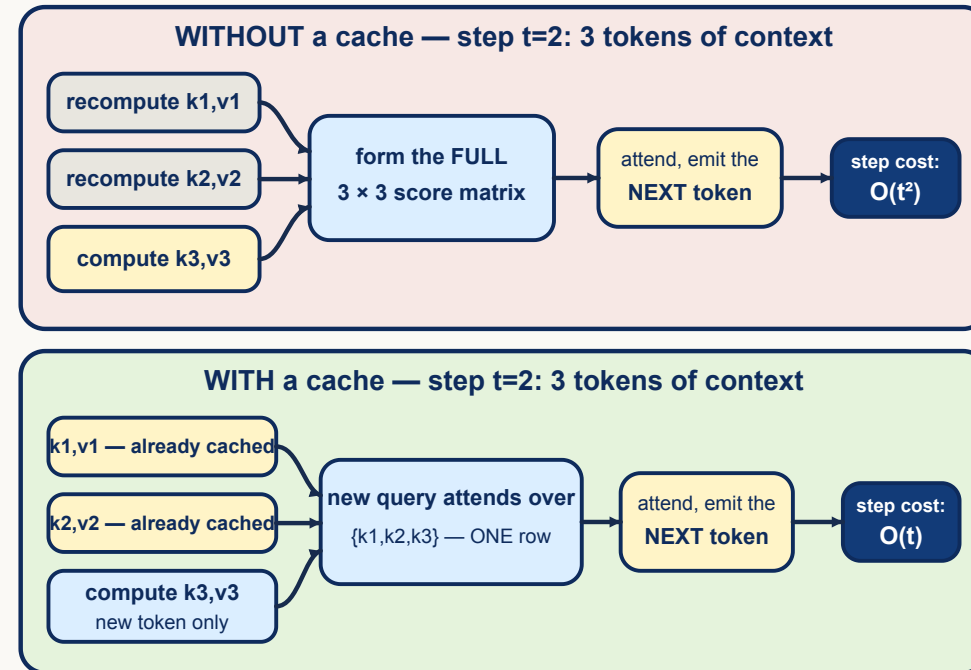
⚠ **Misconception:** *"FlashAttention approximates attention."*
It computes attention **exactly** — same output — and only reorders the computation.



The KV cache — decoding keeps recomputing what it already knows

Under the causal mask, earlier keys and values **can never change**. Recomputing them at every generation step is pure waste.

A **key-value (KV) cache** stores each K, V once. A new step computes only the new token's q, k, v ; its query attends to all cached keys.

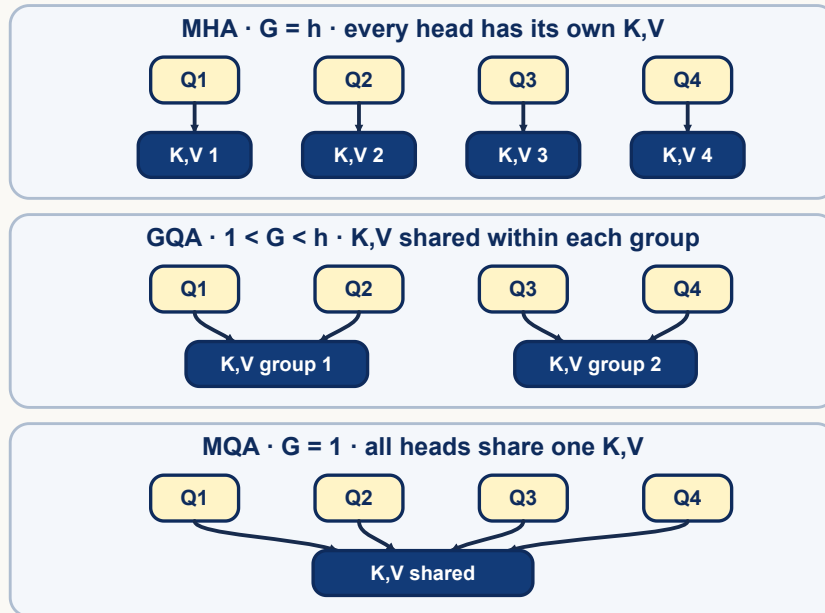


Made concrete: a toy 3-token continuation

Step t — context is $t+1$ tokens	Without a cache	With a cache
$t = 0$	compute q, k, v for the 1 token so far	same — nothing to reuse yet
$t = 1$	recompute q, k, v for both tokens, then attend	reuse token 1's cached k, v ; compute only token 2's q, k, v ; attend the new query over $\{k_1, k_2\}$
$t = 2$	recompute q, k, v for all three tokens, then attend	reuse tokens 1–2's cached k, v ; compute only token 3's q, k, v ; attend over $\{k_1, k_2, k_3\}$

The "without" column keeps doing work it already did at the previous step; the "with" column never repeats itself.

Why GQA/MQA exist — shrinking the cache



Ordinary multi-head attention caches a **separate** K, V per head — memory scales with $d_{\text{model}} = h d_k$. Sharing K, V across a group replaces that with $G d_k$: exactly a h/G **shrink**.

Ainslie et al. (2023) convention — G counts the **KV groups**:

- $G = h \rightarrow$ **MHA**
- $1 < G < h \rightarrow$ **GQA** (LLaMA-2/3)
- $G = 1 \rightarrow$ **MQA**

⚠ The book's own G runs the **opposite** way — we teach **Ainslie's convention only**, never mixed.

Part III — What is the model actually telling you?

A Transformer is not interpretable for free

Nothing in the architecture explains *why* a prediction came out the way it did. Two families of methods try:

Attention-specific

- Peek at the weights the model **already computes**.
- Cheap, and *suggestive* of where the model looked.

Model-agnostic

- Treat the model as a **black box** and probe it from outside.
- Work on *any* model — and are often **more faithful**.

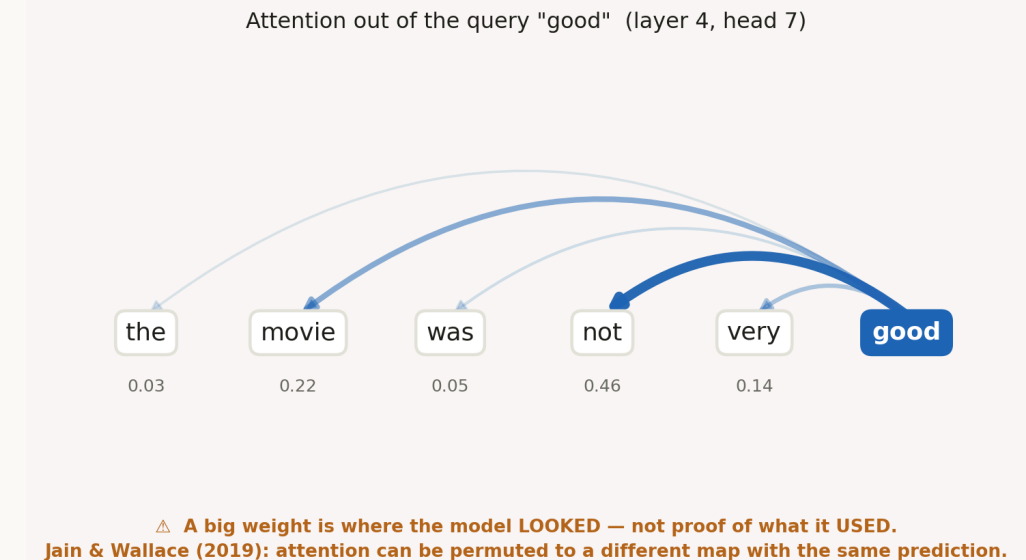
! Misconception: *"Only attention weights can explain a Transformer."* LIME, TCAV and TracIn need no access to attention at all — and several answer a better-posed question.

(a) Attention maps and saliency

Attention maps — visualize the matrix A : which tokens does each query attend to? Drawn as a **heatmap** or as **token-to-token arcs** (the `bertviz` view used in the labs).

Saliency asks: *if I change this token a little, how much does the prediction change?*

- A large change → **high saliency**.
- Almost no change → **low saliency**.



Paper spotlight — *Attention is not Explanation*

Jain & Wallace (2019), NAACL — arXiv:1902.10186

The finding

- You can often construct a **different** attention distribution that yields the **same** prediction.
- If two different "explanations" produce identical outputs, neither can be *the* reason.

The consequence

- A heatmap shows **where** attention mass sat, not **why** the model decided.
- Use attention maps to *generate hypotheses*; verify with a method that actually perturbs the input.

! Misconception: *"High attention weight = the model used that token as its reason."*

(b) Model-agnostic methods — a quick tour

Method	Answers	How, in one line
LIME	why <i>this</i> prediction?	change parts of the input and observe what changes the prediction
TCAV	does a human <i>concept</i> matter?	test whether moving toward a concept changes the prediction
TracIn	which <i>training examples</i> influenced this?	compare the prediction with examples seen during training

Survey depth — the primary papers carry the algebra (Ribeiro 2016, Kim 2018, Pruthi 2020).

→ To the lab: layer tricks, cost, and what the model is (and isn't) telling you

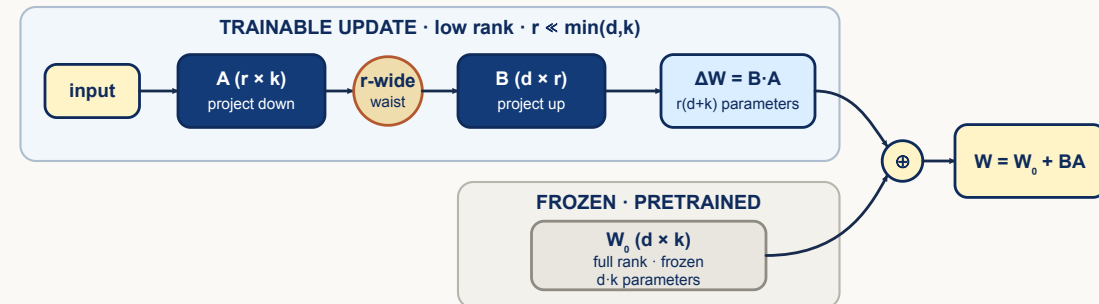
`lab-4-tricks.ipynb` puts today's three threads in your hands.

- **E8 — what do LayerNorm and the residual actually buy?** Undo LN on a batch (where one (γ, β) can't undo every row), then sweep depth $\times$ mode $\in \{\text{none, post, pre}\}$ and measure the gradient norm reaching the input. The **robust** finding is the no-residual collapse; **"I couldn't separate pre- vs post-LN at this scale" is a fully acceptable answer** if backed by your own numbers.
- **E9 — what does the cache actually buy, and cost?** Sweep the naive-vs-cached generation cost, the KV-cache memory footprint across context lengths, and recover GQA/MQA's h/G shrink exactly.
- **E10 — does high attention mean high contribution?** Build your own toy configurations where attention weight and gradient saliency **agree**, and where they **disagree** — and explain the disagreement via softmax saturation.

LoRA — fine-tuning without touching the weights

Full fine-tuning updates **every** entry of **every** weight matrix. But the *change* a downstream task needs, ΔW , tends to need far fewer independent directions than W has — a **low-rank** update is often enough.

Low-Rank Adaptation (LoRA): freeze the pretrained W_0 , and represent the *update* as a product of two thin matrices, $\Delta W = BA$. Train only B, A ; W_0 never moves.



Recap (1/2)

- **Residual (1):** $\partial y / \partial x = I + \partial f / \partial x$ — the identity term is never multiplied down, so deep stacks still train.
- **LayerNorm (2)** standardizes each token over its own features; γ, β can undo it. **Pre- vs post-LN is a real design choice** (placement, warmup) — but the cleanly measurable lab result is the **no-residual ablation**, not a clean pre/post separation.
- **The FFN (3)** is per-token nonlinear muscle; **label smoothing (4)** softens the target, not the model.

Recap (2/2)

- The $O(n^2)$ wall has three exits: **sparse** and **low-rank** are *approximate*; **FlashAttention** is *exact* and reduces memory traffic.
- The **KV cache** avoids redundant recomputation; its memory cost motivates **GQA/MQA**.
- **Attention is not explanation**: treat maps as hypotheses and verify them with model-agnostic methods.
- **LoRA (5)**: freeze W_0 and train $B \cdot A$ — named here, full mechanics left as further reading.

Concept check: Why can LayerNorm placement affect training even if post-LN ultimately normalizes the output? Which lab ablation most clearly shows what the residual path buys?

The whole module, in one arc

- **L1 — QKV + scaled dot-product + multi-head:** attention as a soft lookup; the encoder–decoder assembled and translating.
- **L2 — Anatomy + position:** the four components, the causal mask, and how order gets injected.
- **L3 — The family tree:** BERT / GPT / T5 / mixture-of-experts.
- **L4 — What makes it trainable, affordable, and readable:** layer tricks, speed, interpretability, plus two forward pointers: the KV cache and LoRA.

One equation (L1) grew into one architecture (L1–L2), which forked into a model zoo (L3) — and today closed the loop: how it trains, how it costs, and how (cautiously) it can be read.

References & further reading (1/4)

- [1] A. Amidi and S. Amidi, *Super Study Guide: Transformers and Large Language Models*, Parts 3–4, pp. 82–98, 114–133, and 155–156, 2024.
- [2] K. He *et al.*, “Deep Residual Learning for Image Recognition,” in *Proc. CVPR*, 2016, pp. 770–778.
- [3] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016.
- [4] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” in *Proc. ICML*, 2020, pp. 10524–10533.

References & further reading (2/4)

- [5] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems* 35, 2022.
- [6] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The Efficient Transformer," in *Proc. ICLR*, 2020.
- [7] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," arXiv:2004.05150, 2020.
- [8] S. Wang *et al.*, "Linformer: Self-Attention with Linear Complexity," arXiv:2006.04768, 2020.
- [9] K. Choromanski *et al.*, "Rethinking Attention with Performers," in *Proc. ICLR*, 2021.
- [10] J. Ainslie *et al.*, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proc. EMNLP*, 2023.
- [11] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," in *Proc. MLSys*, 2023.

References & further reading (3/4)

- [12] S. Jain and B. C. Wallace, "Attention Is Not Explanation," in *Proc. NAACL-HLT*, 2019, pp. 3543–3556.
- [13] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic Attribution for Deep Networks," in *Proc. ICML*, 2017, pp. 3319–3328.
- [14] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in *Proc. KDD*, 2016, pp. 1135–1144.
- [15] B. Kim *et al.*, "Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors," in *Proc. ICML*, 2018.

References & further reading (4/4)

- [16] G. Pruthi *et al.*, "Estimating Training Data Influence by Tracing Gradient Descent," in *Advances in Neural Information Processing Systems* 33, 2020.
- [17] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. ICLR*, 2022.
- [18] J. Vig, "A Multiscale Visualization of Attention in the Transformer Model," in *Proc. ACL: System Demonstrations*, 2019, pp. 37–42.